

AMD Unsloth GRPO Fine-tuning

Malini Bhandaru | Shailen Sobhee

Agenda

01

Fine-tuning: what and when

The problem it solves

02

Types of fine-tuning

LoRA, QLoRA, SFT, GRPO

03

The Unsloth advantage

Faster, leaner, no-code

04

A game of Sudoku

Our reward-driven task

05

AMD Radeon GPU

rocm-smi and rocminfo

06

Hands-on notebook

Run it yourself



Part 1: The Concepts

What a model is, what fine-tuning changes, and how GRPO learns from rewards

What is a Large Language Model?

A neural network trained on a huge amount of text so it can predict **the next token** given what came before.

That one simple skill, done at enormous scale, is enough to chat, summarise, write code and reason.

● ● ● next-token prediction

prompt > The capital of France is

Paris 0.94

a city 0.03

located 0.01

Base models

Raw next-token predictors. Great starting point for training, awkward to chat with.

Instruct models

Further trained to follow instructions and hold a conversation.

Generalists

Llama, Qwen, Mistral on Hugging Face know a bit of everything, your job in particular: no.

What is fine-tuning? Why do it?

Take a pre-trained model and keep training it a little more on **YOUR** data, so it adapts to **YOUR** task, domain, tone or format.

1

Voice

Speak in a specific style or persona: your brand, your support agent.

2

Domain

Know legal, medical, or your own product documentation.

3

Format

Always return JSON. Always cite sources. Every time.

4

Focus

Do one narrow task extremely well, cheaply, on a small model.

Often cheaper and more reliable than stuffing everything into a giant prompt, or paying for an even bigger model. **It lets a small model punch above its weight.**

LoRA and QLoRA

Parameter-efficient fine-tuning: freeze the giant model, train a tiny adapter.

LoRA

Low-Rank Adaptation

- Inject small low-rank matrices into chosen layers
- Original weights stay frozen, only adapters train
- Trainable parameters drop by orders of magnitude

QLoRA

Quantized Low-Rank Adaptation

- Everything LoRA does, plus a 4-bit base model
- Cuts VRAM again, so bigger models fit on one GPU
- The usual answer to an out-of-memory error

~0.1%

of parameters trained

4-bit

base weights under QLoRA

Comparable

quality to full fine-tuning

Training methodology: SFT and GRPO

Two ways to tell a model it did well. One shows the answer, the other scores attempts.

SFT

Supervised Fine-Tuning

- Uses labelled data: prompt in, correct answer out
- The model imitates the answers you supply

```
{"instruction": "Summarize in one line.",  
  "input": "<article text>",  
  "output": "<summary>"}
```

GRPO

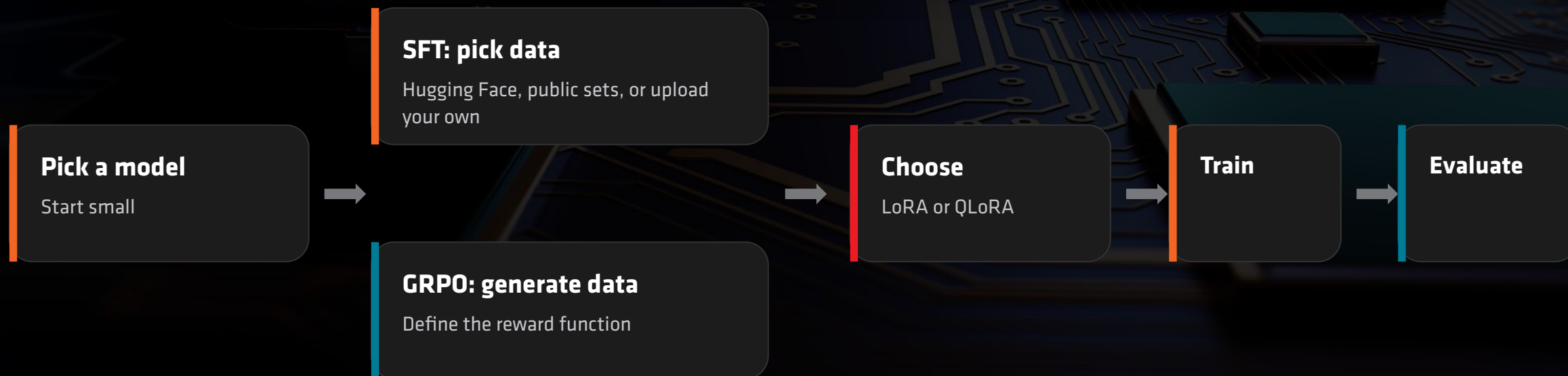
Group Relative Policy Optimization

- No labelled answer needed, only a way to score one
- Model generates a group of attempts and compares them

- 1 **Generate** several candidate answers
- 2 **Score** each with a reward function
- 3 **Prefer** the ones that scored above the group

The fine-tuning pipeline

Same spine either way. The data step is what differs.



Training knobs

learning rate

number of steps

time limit

max sequence length

Unslloth

An open-source, no-code framework and UI for training, running and exporting LLMs **locally**, tuned for speed, memory efficiency and accessibility.

It sits as an optimized layer on top of Hugging Face Transformers, so your existing workflow still applies.

LoRA and QLoRA

built in, no glue code

Quantization

4-bit and 8-bit paths

Optimized kernels

faster attention and MLP

Consumer hardware

one GPU is enough

In this workshop

Unslloth is what makes a GRPO run finish inside a session slot instead of overnight.



Part 2: Hands-On

A game of Sudoku, an AMD Radeon GPU, and a notebook you drive yourself

A game of Sudoku

Sudoku means “**single number**” in Japanese.

A logic puzzle: place digits 1 to 9 so every row, every column and every 3x3 box contains each digit exactly once.

Verifiable

A board is right or wrong. No human judge needed.

Reward-friendly

Score partial credit per correct cell, row and box.

Genuinely hard

A general model gets it wrong. Improvement is visible.

easy

00:41

4		2	9		5	3	7	
6		9		4				
5	8	7		2	6			9
7	9				1	2	4	8
			2	3			6	
								3
1				9		6		4
9		6	8	1		5	3	
	4			6		7		1

Our training task: an easy grid, scored cell by cell

AMD Unsloth fine-tuning workshop

1

Open the portal

notebooks.amd.com/ind-dev

2

Enter the password

[RadeonUnslothTuning](#)

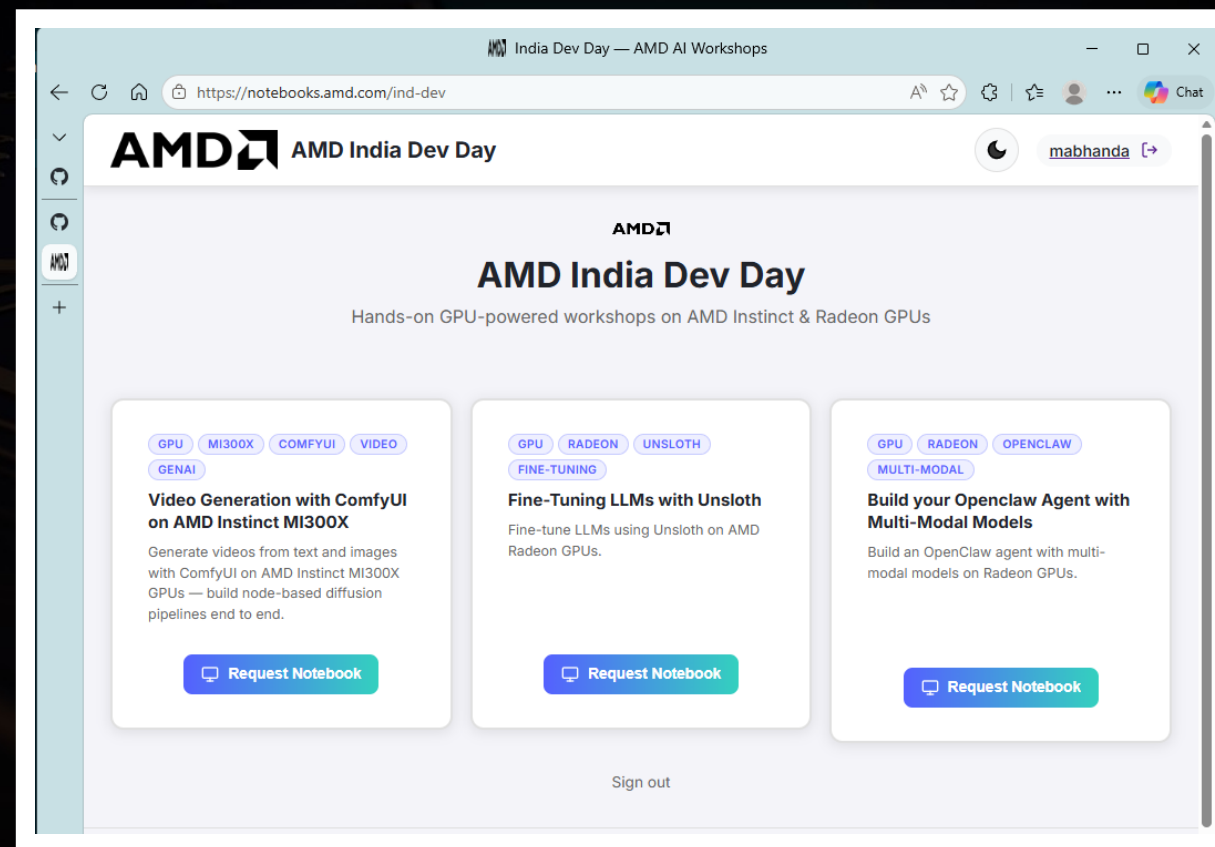
3

Pick the Unsloth card

Lands you on an AMD Radeon GPU

It is a real GPU, not a simulator

Hosted Jupyter, nothing to install locally



GPU visibility and health

Open a terminal inside Jupyter and confirm the GPU is really there before you train.

terminal

```
$ rocm-smi
```

```
$ rocminfo
```

rocm-smi

Live view: temperature, power, clocks, VRAM use

rocminfo

Static view: agents, ROCm version, GPU identity

GPU busy at 0% while a cell runs? It is not training.

```
root@hf-11403-00249838:/workspace/template-repos/repo-39d0cf1ec4f9/repo# rocm-smi
```

```
===== ROCm System Management Interface =====
=
===== Concise Info =====
=
Device  Node  IDs              Temp    Power  Partitions          SCLK  MCLK  Fan   Perf  PwrCap  VRAM%  GPU%
              (DID,      GUID)    (Edge)  (Avg)  (Mem, Compute, ID)
=====
0        3    0x744b,  49884  27.0°C  7.0W   N/A, N/A, 0        0Mhz  96Mhz  20.0% auto  241.0W  0%    0%
=====
===== End of ROCm SMI Log =====
root@hf-11403-00249838:/workspace/template-repos/repo-39d0cf1ec4f9/repo#
```

```
root@hf-11403-00249838:/workspace/template-repos/repo-39d0cf1ec4f9/repo# rocminfo
ROCK module version 6.16.13 is loaded
=====
HSA System Attributes
=====
Runtime Version:      1.18
Runtime Ext Version:  1.15
System Timestamp Freq.: 1000.000000MHz
Sig. Max Wait Duration: 18446744073709551615 (0xFFFFFFFFFFFFFFFF) (timestamp count)
Machine Model:        LARGE
System Endianness:    LITTLE
MmioX:                DISABLED
XNACK enabled:        NO
DMAbuf Support:       YES
VMM Support:          YES

=====
HSA Agents
=====
Agent 1
=====
Name:                 AMD EPYC 9334 32-Core Processor
Uuid:                 CPU-XX
Marketing Name:        AMD EPYC 9334 32-Core Processor
Vendor Name:          CPU
Feature:               None specified
Profile:               FULL PROFILE
```

rocminfo confirms the ROCm runtime version and every agent the host exposes, GPU and CPU alike.

Driving the notebook

Run cells top to bottom. The first model load is the slow one.

Watch the marker

In [*] means still running, In [7] means done

Do not skip ahead

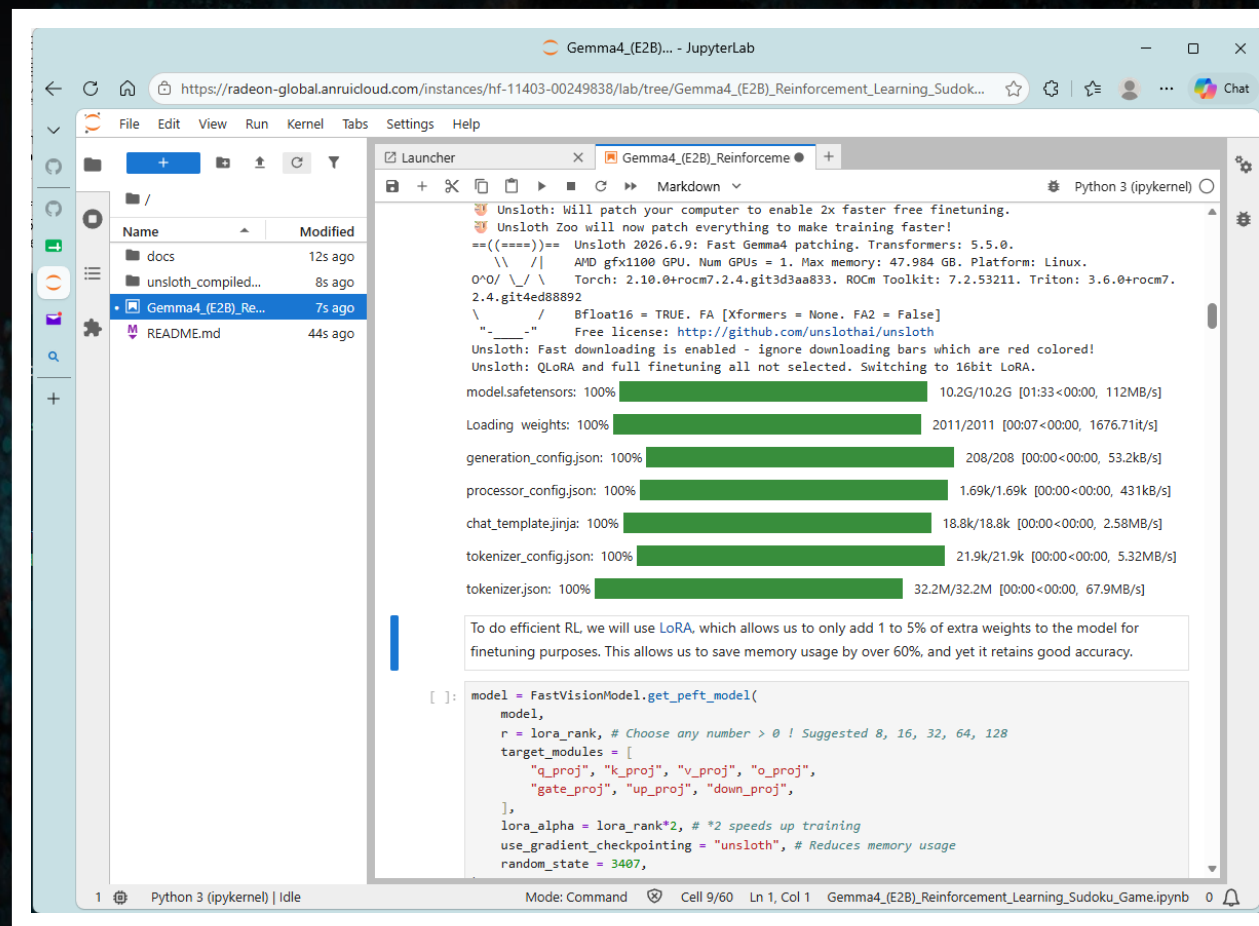
Later cells depend on variables the earlier ones define

Keep an eye on the GPU

Second terminal tab running `rocm-smi`

Cold start is normal

First load pulls weights. Retry a timed-out chat, the warm model is fast



Gemma4_(E2B)... - JupyterLab

https://radeon-global.anruicloud.com/instances/hf-11403-00249838/lab/tree/Gemma4_(E2B)_Reinforcement_Learning_Sudoku...

File Edit View Run Kernel Tabs Settings Help

Launcher

Python 3 (ipykernel)

Unsloth: Will patch your computer to enable 2x faster free finetuning.
Unsloth Zoo will now patch everything to make training faster!
Unsloth 2026.6.9: Fast Gemma4 patching. Transformers: 5.5.0.
AMD gfx1100 GPU. Num GPUs = 1. Max memory: 47.984 GB. Platform: Linux.
Torch: 2.10.0+rocm7.2.4.git3d3aa833. ROCm Toolkit: 7.2.53211. Triton: 3.6.0+rocm7.2.4.git4ed88892
Bfloat16 = TRUE. FA [Xformers = None, FA2 = False]
Free license: <http://github.com/unslothai/unsloth>
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
Unsloth: QLoRA and full finetuning all not selected. Switching to 16bit LoRA.

models.safetensors: 100% 10.2G/10.2G [01:33<00:00, 112MB/s]
Loading weights: 100% 2011/2011 [00:07<00:00, 1676.71it/s]
generation_config.json: 100% 208/208 [00:00<00:00, 53.2kB/s]
processor_config.json: 100% 1.69k/1.69k [00:00<00:00, 431kB/s]
chat_template.jinja: 100% 18.8k/18.8k [00:00<00:00, 2.58MB/s]
tokenizer_config.json: 100% 21.9k/21.9k [00:00<00:00, 5.32MB/s]
tokenizer.json: 100% 32.2M/32.2M [00:00<00:00, 67.9MB/s]

To do efficient RL we will use LoRA, which allows us to only add 1 to 5% of extra weights to the model for finetuning purposes. This allows us to save memory usage by over 60%, and yet it retains good accuracy.

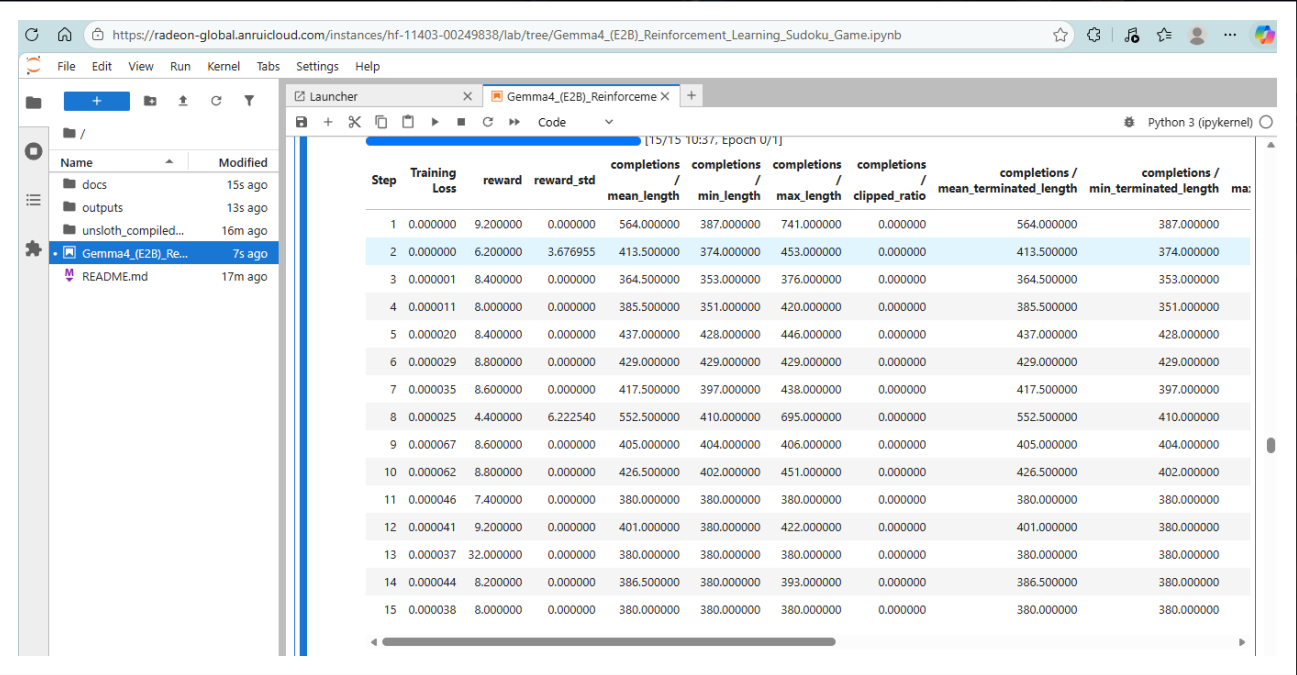
```
[ ]: model = FastVisionModel.get_peft_model(
    model,
    r = lora_rank, # Choose any number > 0 ! Suggested 8, 16, 32, 64, 128
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha = lora_rank*2, # *2 speeds up training
    use_gradient_checkpointing = "unsloth", # Reduces memory usage
    random_state = 3407,
```

1 Python 3 (ipykernel) | Idle Mode: Command Cell 9/60 Ln 1, Col 1 Gemma4_(E2B)_Reinforcement_Learning_Sudoku_Game.ipynb 0

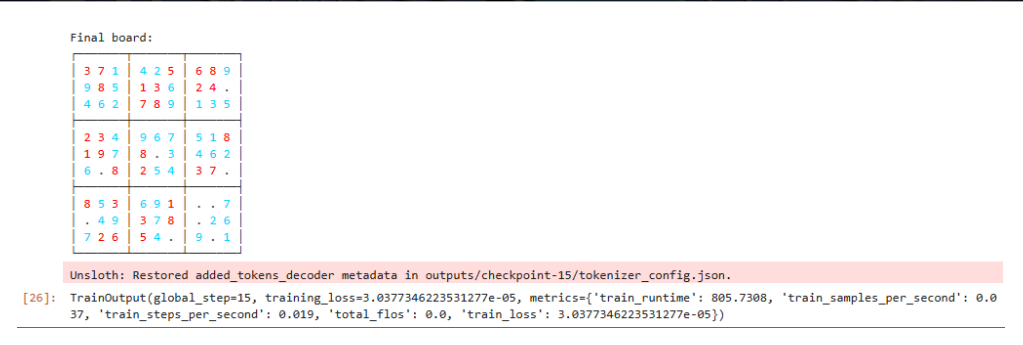
Fine-tuning complete

Fifteen GRPO steps on a Radeon GPU, and the reward curve moves.

DEMO: 15 STEPS



Per-step training loss and reward, straight from the notebook



Final board produced by the tuned model

What to look for

- Reward trending up, not the absolute value
- A structurally valid board, no repeated digits

Troubleshooting

SYMPTOM

FIX

First chat response times out

Cold model load. Retry, the warm model answers fast.

Training data format issues

Check the Alpaca fields: instruction, input, output.

Out of memory during training

Smaller model, enable QLoRA 4-bit, lower batch size or sequence length.

Loss not decreasing

Raise the learning rate a little, train more steps, re-check the dataset format.

Cell never completes

Run rocm-smi. GPU use drops to zero when training is actually finished.

Need a first experiment idea?

Start with SFT on a Hugging Face set, then GRPO on math where answers are checkable.

Learn more

Unsloth

github.com/unslothai/unsloth

The framework itself

Unsloth docs

docs.unsloth.ai

Guides, notebooks, supported models

LoRA paper

arxiv.org/abs/2106.09685

Low-Rank Adaptation, Hu et al.

QLoRA paper

arxiv.org/abs/2305.14314

4-bit finetuning, Dettmers et al.

GRPO / DeepSeekMath

arxiv.org/abs/2402.03300

Where GRPO is introduced

Workshop notebook

notebooks.amd.com/ind-dev

Password: RadeonUnslothTuning

AMD

